LYCEUM OF THE PHILIPPINES UNIVERSITY – LAGUNA

BDAT05 — Fundamentals of Predictive Analytics · Term 1, AY 2026–2027

First Contact — Session 3 Lab Sheet

Saturday 29 August · work in Colab, record in pen · your Part C output IS your P2, committed before 5:00 PM

Name: Sanchez, Eliah Therrien E. Track: P · _____

Ground rules: datasets load from Drive, never get committed to the repo. Every cleaning decision goes in your log WITH its reason. When code errors, read the message bottom-up before raising a hand — then raise it. The column names below are placeholders: your FIRST job in every task is to discover the real names with `df.info()` and adapt.

PART A · GUIDED, EVERYONE

First contact (with the class, 1:35–2:10)

A1 Load and shake hands · 8 min

```
import pandas as pd
sales = pd.read_csv("<path>/sales.csv")   # adapt path & filename
sales.info()
sales.head(10)
```

1. Rows: **10589** Columns: **9**. Write THREE things you notice (a strange dtype, a missing count, a column you did not expect):

- The Date column has missing values.

- The CustomerID column has a lot of missing values.

- Range Index has 10,589 entries with the rows numbered from 0 to 10,588

A2 The dtype hunt · 8 min

2. Find at least one column whose dtype is WRONG for what it holds (a number stored as object, a date stored as text). Name it, and say what the wrong type would silently break:

- The DATE column has the wrong type because it is stated as object instead of a date.

A3 Diagnostic Q8, at last · 9 min

```
sales.groupby("<branch_col>")["<amount_col>"].sum().sort_values(ascending=False)
```

3. Which branch leads? Does the ranking match your Session-2 intuition about the six branches? One sentence:

- B01 leads with total sales of: 607,298.18. The ranking is mostly what I expected, but some branch names are written differently.

SQL twin (read, don't run): `SELECT branch, SUM(amount) FROM sales GROUP BY branch ORDER BY 2 DESC;`

A4 The join — and the first orphans · 10 min

```
merged = pd.merge(sales, branches, on="<branch_key>", how="inner")
len(sales), len(merged)   # compare!
```

4. Row count before: **10589** after: **10583**. If they differ, rows vanished in the join — orphan keys. How many, and what might they be?

- 6 rows vanished. These may be caused by branch IDs that do not match between the two tables.

PART B · GUIDED, EVERYONE

The validation script (3:00–3:20, after break)

We build this function together — it becomes the heart of your P2 validation report. Five checks, straight from the syllabus: type, range, uniqueness, completeness, referential.

```
def validate(df, name):
    print(f"=== {name} ===")
    print("rows:", len(df))
    print("dtypes:\n", df.dtypes)                    # 1 type
    print("missing:\n", df.isna().sum())             # 2 completeness
    print("duplicate ids:", df["<id_col>"].duplicated().sum())   # 3 uniqueness
    print("negative amounts:", (df["<amount_col>"] < 0).sum())   # 4 range
```

```
    # 5 referential: keys in this table missing from the parent table
    # orphans = set(df["<key>"]) - set(parent["<key>"])
```

5. Run it on the sales table. Record your two most interesting findings (these files are imperfect ON PURPOSE — what you find here is the graded point, not a distraction):

- There are 4,775 missing CustomerID values.

- There are 23 duplicate SaleID values.

PART C · INDIVIDUAL — YOUR P2 SLICE

Your canvas meets your data (3:35–4:35)

Each of you now builds the table your P1 promised — YOUR grain, from YOUR files. Find your track below; everything after the table is common to all four.

Track	Build this slice	Your P1 connection
P4 · MKT Anthonne	Branch-month revenue: total sales amount per branch per month (merge sales + branches; derive month from the date column).	Your field 4, made real. Decide monthly ONCE — the wobble ends today. Count your rows.
P5 · FIN Eliah	Product-month revenue: total sales amount per product per month (merge sales + products).	Your grain fix, executed. Count rows at product-YEAR, then product-MONTH — write both numbers and feel the difference.
P6 · HR Angus	Branch-month workforce table: headcount, and average tenure per branch per month (employees table + calendar).	Your P1 seed. NOTE: employee data is SENSITIVE BY DEFAULT — your Part C-4 lawful-basis paragraph carries extra weight, and no names ever appear in outputs.
P7 · OPS Lawrence	Branch-month order counts: number of orders per branch per month (sales + calendar).	Your field 4 exactly — branch-month, ~108 rows. Verify that count for real.

C1 Build and count · 20 min

1. Build the slice. Rows in YOUR table: **224**. Does the count match the grain your (revised) P1 promised? One sentence:\

- Rows in my table: 224. Yes, this matches the product-month grain I promised because each row represents one product for one month.

C2 Data dictionary · 10 min

For every column in your slice: name · dtype · plain-language meaning · allowed values/range · source file(s). Start here, finish in the notebook:

Column	dtype	Meaning	Allowed / range
Product ID	object	ID of the product	Product IDs from the Products file
Year	int64	Year of the sale	2025–2026
Month	int64	Month of the sale	1–12
Amount	float64	Total sales amount for the product in that month	0 or higher

C3 Validate and log · 15 min

2. Run validate() on your slice. Log at least THREE findings, and for each write the cleaning DECISION and its REASON (issue → decision → why). This is your cleaning log — the provenance record Gate 1 depends on:

- Year and Month are stored as floats, but I kept them because they still show the correct year and month.

- There are no missing values, so I kept all 224 rows.

- There are no duplicate product-month records, so I kept all the rows.

C4 Lawful basis statement · 8 min

Complete in your notebook, 3–5 sentences: *What data does this slice use? Does any of it identify a person? If no — say why the aggregation protects it. If yes — state the lawful basis (RA 10173), how it is de-identified, and who may see it.* **(Angus: yours must also state that employee data is sensitive by default and that no individual is identifiable in any output.)**

C5 Package and commit · 7 min + the commit block

P2 checklist — all five, in `<your-folder>/p2-data-preparation/`: ☐ notebook (runs top-to-bottom from a clean copy — restart & Run all before committing) ☐ data dictionary ☐ validation findings ☐ cleaning log with reasons ☐ lawful basis statement. Cleaned data: Drive link in the folder README — never a committed CSV. Commit message: "`P2: <surname> — slice, dictionary, validation v1`". Content freezes at 5:00 PM; copy-editing only until 11:59 tonight.

The point of today, in one line: last week you promised a table (P1). Today you built it, counted it, doubted it, and documented it (P2). Next week you interrogate it (P3).